

Lab Task No. 02

Course Title: CS-426L Parallel and Distributed Computing Lab

TITLE OF THE TASK

"Implementation and Practical Applications of Array-Based Stacks and Queues in Java"

SUBMITTED BY:

Name: Usama Arshad

Student ID: 247141

Semester: 5th

Section: BS-CYS-F24-A

SUBMITTED TO:

Instructor Name: Mr. Qaiser Ali

Date of Submission: September 26, 2026

AIR UNIVERSITY, KHARIAN CAMPUS

DEPARTMENT OF Computer Science

Question 01:

You are a systems engineer building the core execution engine for an enterprise transaction processor. To maintain data consistency during system recovery, transactional operations must be recorded in contiguous, high-speed memory using an array-backed memory model before being committed or inspected in reverse chronological order. Design and implement an array-based Stack engine in Java that simulates transaction logging. Your program should ingest a series of integer-based transaction IDs, push them onto the stack using an underlying array implementation, and then unwind the stack to print the execution trace on the console to verify system state.

Code:

```
class TransactionStack {
    private int[] stackArray;
    private int top;
    private int capacity;

    // Constructor to initialize stack
    public TransactionStack(int size) {
        capacity = size;
        stackArray = new int[capacity];
        top = -1; // -1 indicates an empty stack
    }

    // Push operation to add a transaction ID
    public void push(int transactionId) {
        if (top == capacity - 1) {
            System.out.println("Stack Overflow: Cannot log transaction " + transactionId);
            return;
        }
        stackArray[++top] = transactionId;
        System.out.println("Logged Transaction: TXN_" + transactionId);
    }

    // Pop operation to unwind the stack
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow: No transactions to unwind.");
            return -1;
        }
        return stackArray[top--];
    }

    // Check if the stack is empty
    public boolean isEmpty() {
        return top == -1;
    }
}

public class TransactionProcessor {
```

```
    public static void main(String[] args) {
        TransactionStack stack = new TransactionStack(5);

        // 1. Ingest transaction IDs
        System.out.println("--- Pushing Transactions onto Stack ---");
        stack.push(1001);
        stack.push(1002);
        stack.push(1003);
        stack.push(1004);

        // 2. Unwind stack in reverse chronological order
        System.out.println("\n--- Unwinding Execution Trace (LIFO) ---");
        while (!stack.isEmpty()) {
            int txn = stack.pop();
            System.out.println("Recovered / Rolled Back: TXN_" + txn);
        }
    }
}
```

Output:

```
--- Pushing Transactions onto Stack ---
Logged Transaction: TXN_1001
Logged Transaction: TXN_1002
Logged Transaction: TXN_1003
Logged Transaction: TXN_1004

--- Unwinding Execution Trace (LIFO) ---
Recovered / Rolled Back: TXN_1004
Recovered / Rolled Back: TXN_1003
Recovered / Rolled Back: TXN_1002
Recovered / Rolled Back: TXN_1001
```

Explanation:

I created an array-based stack to manage transaction IDs, utilizing stackArray, top, and capacity for efficient storage. I initialized top to -1 in the constructor, ensuring the stack begins empty. I implemented push() and pop() methods, complete with checks for overflow and underflow. In the push() method, I incremented top and added the transaction ID, while in pop(), I returned the top value and decremented top, adhering to the Last In, First Out (LIFO) principle. In the main function, I set up a stack with a capacity of 5, pushed four transaction IDs (1001 to 1004), and subsequently popped them off until isEmpty() returned true. The output reflects the order of operations: it first shows the pushes, then displays the unwinding in reverse order, demonstrating that the last transaction pushed is the first one to be rolled back.

Question 02:

In modern cybersecurity architectures, certain zero-trust wire protocols utilize mirrored data payloads to ensure packet integrity across high-latency links. To audit incoming byte-streams without memory allocation overhead, edge security gateways compare LIFO and FIFO processing streams simultaneously. Develop a signature verification engine in Java that ingests an incoming data string payload. Your system must push each character of the string into a Stack while simultaneously enqueuing the characters into a Queue. By popping from the stack and dequeuing from the queue concurrently, determine whether the payload exhibits bidirectional symmetry (a palindrome) and log the verification result.

Code:

```
// Array-based Stack for characters
class CharStack {
    private char[] arr;
    private int top;

    public CharStack(int capacity) {
        arr = new char[capacity];
        top = -1;
    }

    public void push(char ch) {
        arr[++top] = ch;
    }

    public char pop() {
        return arr[top--];
    }
}

// Array-based Queue for characters
class CharQueue {
    private char[] arr;
    private int front;
    private int rear;

    public CharQueue(int capacity) {
        arr = new char[capacity];
        front = 0;
        rear = -1;
    }

    public void enqueue(char ch) {
        arr[++rear] = ch;
    }

    public char dequeue() {
        return arr[front++];
    }
```

```
}

public class IntegrityVerifier {
    public static void main(String[] args) {
        String payload = "RADAR";
        int len = payload.length();

        CharStack stack = new CharStack(len);
        CharQueue queue = new CharQueue(len);

        // 1. Populate both Stack and Queue simultaneously
        for (int i = 0; i < len; i++) {
            char ch = payload.charAt(i);
            stack.push(ch);
            queue.enqueue(ch);
        }

        // 2. Compare LIFO (stack) and FIFO (queue) outputs
        boolean isSymmetric = true;
        for (int i = 0; i < len; i++) {
            char fromStack = stack.pop();
            char fromQueue = queue.dequeue();

            if (fromStack != fromQueue) {
                isSymmetric = false;
                break;
            }
        }

        // 3. Log results
        System.out.println("Inspected Payload: " + payload);
        if (isSymmetric) {
            System.out.println("Verification SUCCESS: Payload exhibits bi-directional symmetry.");
        } else {
            System.out.println("Verification FAILED: Payload is asymmetric.");
        }
    }
}
```

Output:

```
Inspected Payload: RADAR
Verification SUCCESS: Payload exhibits bi-directional symmetry.
```

Explanation:

I developed two classes, CharStack and CharQueue, which utilize arrays to manage characters. The stack operates on a Last-In, First-Out (LIFO) basis, using push and pop methods, while the queue follows a First-In, First-Out (FIFO) structure with enqueue and dequeue methods. In the main function, I took the string payload **RADAR** and pushed each character onto the stack, simultaneously enqueuing it into the queue. Then, as I popped characters

from the stack—reversing the order—and dequeued them from the queue—yielding the original order—I compared each pair to verify symmetry. If all pairs matched, I set isSymmetric to true; otherwise, I exited early from the loop. Finally, I printed the result of the verification, and for **RADAR**, it turned out successful, confirming that the word reads the same backward as it does forward.

Question 03:

High-throughput backend systems experience bursty traffic where millions of user requests hit system endpoints simultaneously. To prevent service crashes, microservices ingest incoming raw request batches from an array into an asynchronous message broker, buffering tasks so worker threads can process them strictly in order of arrival. Implement a message queue buffer component in Java. Given an incoming array arr[] of size *N* representing raw event task payloads, write a program that enqueues all array elements sequentially into a custom Queue buffer, and subsequently dequeues and processes each element to simulate orderly task consumption.

Code:

```
class CustomQueue {
    private int[] data;
    private int front;
    private int rear;
    private int size;
    private int capacity;

    public CustomQueue(int capacity) {
        this.capacity = capacity;
        data = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }

    // Insert task into buffer
    public void enqueue(int task) {
        if (size == capacity) {
            System.out.println("Buffer Overflow: Cannot enqueue task " + task);
            return;
        }
        rear++;
        data[rear] = task;
        size++;
    }

    // Process task from buffer
    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Buffer Underflow: No tasks to process.");
            return -1;
        }
```

```
        int task = data[front];
        front++;
        size--;
        return task;
    }

    public boolean isEmpty() {
        return size == 0;
    }
}

public class MessageQueueBuffer {
    public static void main(String[] args) {
        // Raw incoming request batch
        int[] rawBatch = {501, 502, 503, 504, 505};
        int n = rawBatch.length;

        CustomQueue buffer = new CustomQueue(n);

        // 1. Enqueue elements from the array sequentially
        System.out.println("--- Buffering Incoming Tasks ---");
        for (int i = 0; i < n; i++) {
            System.out.println("Enqueuing task: " + rawBatch[i]);
            buffer.enqueue(rawBatch[i]);
        }

        // 2. Dequeue and consume tasks in FIFO order
        System.out.println("\n--- Processing Tasks in Arrival Order ---");
        while (!buffer.isEmpty()) {
            int task = buffer.dequeue();
            System.out.println("Worker consumed task: " + task);
        }
    }
}
```

Output:

```
--- Buffering Incoming Tasks ---
Enqueuing task: 501
Enqueuing task: 502
Enqueuing task: 503
Enqueuing task: 504
Enqueuing task: 505

--- Processing Tasks in Arrival Order ---
Worker consumed task: 501
Worker consumed task: 502
Worker consumed task: 503
Worker consumed task: 504
Worker consumed task: 505
```

Explanation:

I developed a CustomQueue class that utilizes an integer array named data, along with variables for front, rear, size, and capacity, to implement a FIFO buffer. In the constructor, I set front to 0, rear to -1, and size to 0, ensuring the queue starts off empty. The enqueue method first checks if size equals capacity to avoid any buffer overflow. If there's space, it increments rear, adds the task to the queue, and then increases size. On the other hand, the dequeue method checks if the queue is empty using isEmpty(), retrieves the value at front, increments front, decreases size, and returns the task. In the main function, I created a queue with a capacity of 5, enqueued tasks ranging from 501 to 505 from rawBatch, and then dequeued them one by one until the queue was empty. The output shows that tasks were processed in the exact order they arrived, confirming the FIFO behavior of the queue.

Question 04:

Legacy network adapters and mainframe interface drivers frequently require string identifiers (such as user handles or session keys) to be byte-reversed before transmission over legacy serial lines. Software engineers use stack-based memory frames to perform fast in-memory string inversions during payload serialization. Construct a protocol header serialization module in Java. Your program must take a target string identifier (e.g., an employee user handle or name), iterate through its individual characters, push them sequentially onto an execution Stack, and then pop each character off the stack to display the reversed string payload on the output terminal.

Code:

```
class StringReversalStack {
    private char[] stackArray;
    private int top;

    public StringReversalStack(int capacity) {
        stackArray = new char[capacity];
        top = -1;
    }

    public void push(char c) {
        stackArray[++top] = c;
    }

    public char pop() {
        return stackArray[top--];
    }

    public boolean isEmpty() {
        return top == -1;
    }
}

public class ProtocolSerializer {
    public static void main(String[] args) {
        String identifier = "USER_SESSION_849";
        int len = identifier.length();
```

```
        StringReversalStack stack = new StringReversalStack(len);

        // 1. Push all characters of the identifier onto the stack
        for (int i = 0; i < len; i++) {
            stack.push(identifier.charAt(i));
        }

        // 2. Pop all characters off the stack to form the reversed string
        String reversedIdentifier = "";
        while (!stack.isEmpty()) {
            reversedIdentifier += stack.pop();
        }

        // 3. Display original and reversed payload
        System.out.println("Original Identifier : " + identifier);
        System.out.println("Reversed for Serial : " + reversedIdentifier);
    }
}
```

Output:

```
Original Identifier : USER_SESSION_849
Reversed for Serial : 948_NOISSES_RESU
```

Explanation:

I developed a StringReversalStack class that utilizes a character array along with a top index initialized to -1. This class includes push, pop, and isEmpty methods to adhere to the Last In, First Out (LIFO) principle. In the main function, I took the identifier **USER_SESSION_849**, created a stack that matched its length, and systematically pushed each character onto the stack. Afterward, I began popping characters off the stack one by one while it wasn't empty, appending each character to reversedIdentifier, which effectively reversed the original string. Finally, I printed both the original and the reversed strings to display the serialized results. The output clearly shows that the last character pushed becomes the first to be popped, resulting in the complete reversal of the identifier.

- End of Lab Task -